HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY
School of Information and Communication Technology

Software Requirement Specification

Version 1

AI-POWERED INSURANCE SIMULATOR

Introduction to Software Engineering

Group Bòng

Nguyễn Hoàng Thái - 20235557

Đặng Xuân Đạt - 20235485

Phạm Lê Minh Quang - 20235554

Phạm Ngọc Thạch - 20235556

Nguyễn Thành Vinh - 20235576

Ha Noi, 4th of May 2026

Table of Contents

1 Introduction

1.1 Document Purpose

This Software Requirements Specification provides a comprehensive and detailed description of the **AI-Powered Insurance Simulator**. It outlines the functional capabilities available at runtime, including the user and group management subsystems, the virtual policy marketplace, the automated parametric claims processing engine, and the AI-driven simulation mode. Furthermore, this document specifies the overall purpose and features of the system, its user and external interfaces, and the structural constraints required for the system to appropriately react to external stimulation.

This document is intended for all project stakeholders, product owners, and the software development team. It serves as the definitive baseline for understanding the system's requirements, guiding both the technical implementation and the evaluation of the final product.

1.2 Scopes

Every robust software system requires a dynamic mechanism for managing users, groups, and functional permissions. Within the context of the AI-Powered Insurance Simulator, the scope encompasses a comprehensive Identity and Access Management subsystem designed to control these elements flexibly at runtime.

Users can register to create their accounts and subsequently authenticate using their local system credentials to access the platform. A standard user is capable of personal profile management.

Administrators possess extensive control over account lifecycles, including the ability to suspend accounts or enforce immediate or periodic password updates. The system utilizes a flexible Role-Based Access Control architecture. Administrators can assign specific roles to users, and a single user can hold multiple roles simultaneously. Each role is configured to grant access to a specific set of system functions. Since multiple roles can utilize the same function, the system supports a highly scalable many-to-many permission matrix.

Upon successful authentication, the application dynamically renders a customized navigation menu based strictly on the user's authorized roles. Selecting an option from this dynamically generated menu securely routes the user to the corresponding functional interface.

1.3 Glossary of Terms

- **Identity and Access Management (IAM):** A comprehensive framework of policies, processes, and technologies that ensures authorized individuals have the appropriate access to system resources. In this platform, it governs user registration, authentication, and secure session management.
- **Role-Based Access Control (RBAC):** A security mechanism within the IAM framework that restricts system access based on the assigned roles of individual users. Instead of assigning permissions to users directly, permissions are assigned to roles, and users are granted these roles, ensuring a scalable and manageable authorization structure.
- **Parametric Insurance:** A non-traditional insurance model that does not indemnify the pure loss, but rather agrees ex-ante to make a predefined payment upon the occurrence of an objective, measurable triggering event (e.g., a flight delay exceeding 120 minutes, or rainfall exceeding 50mm), bypassing manual damage assessment.
- **Trigger Condition:** The specific, mathematically defined parameter or threshold embedded within a virtual policy. When real-time data from an External Data Provider matches or exceeds this threshold, the system automatically initiates the claim evaluation process.
- **External Data Provider:** A third-party system or service (accessed via API) that supplies the platform with real-time, objective data required to evaluate Parametric Insurance triggers.

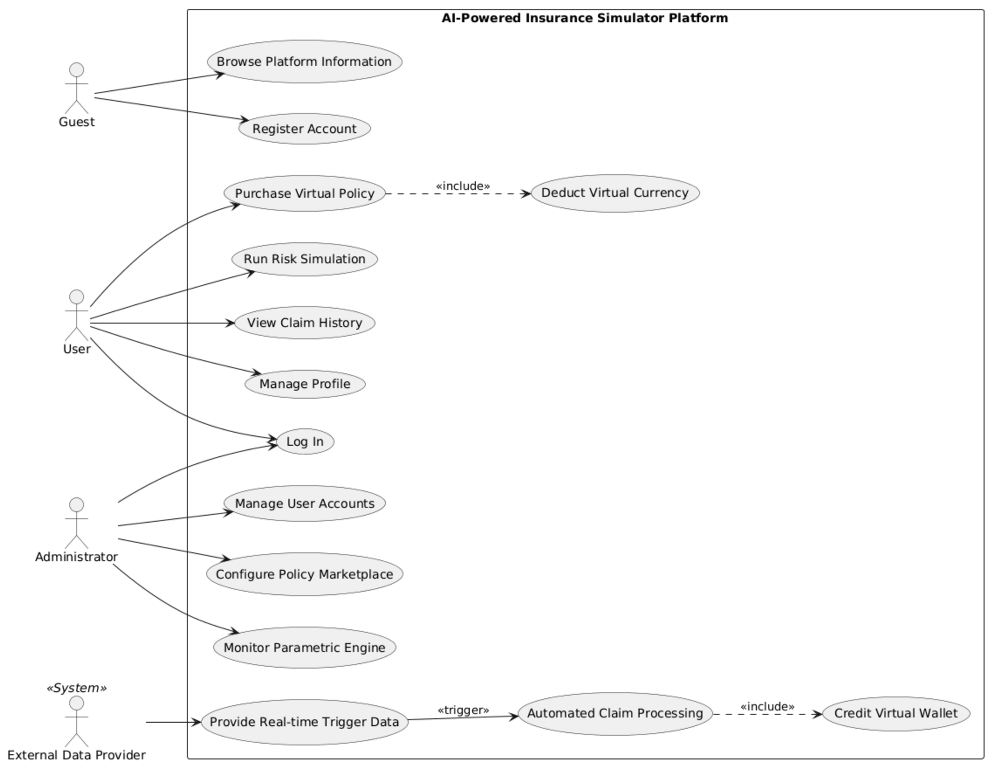
- **API Payload:** The actual data packet transmitted by an External Data Provider to the system. It contains the essential variables and metrics needed by the Parametric Claims Engine to process and validate a potential payout.
- **Virtual Currency / Virtual Wallet:** A simulated financial ledger system isolated within the platform. It allows users to purchase virtual policies and receive automated payouts, enabling risk management education without exposing users to actual financial liabilities.
- **Virtual Insurance Policy:** A digital representation of an insurance contract within the simulation environment. It encapsulates the specific risk coverage, the required virtual premium, the maximum potential payout, and the exact trigger conditions. It is strictly a simulated financial instrument designed for educational purposes and carries no real-world legal or financial obligations.

2 General description

2.1 Actors

The software system interacts with four primary actors. A Guest is an unauthenticated individual browsing the public platform. A User is an authenticated individual purchasing virtual parametric policies and executing risk simulations. An Administrator oversees account access, configures the marketplace, and maintains the core rules of the parametric engine. Finally, an External Data Provider acts as an automated system actor, continuously supplying the real-time feeds required by the claims processing engine to evaluate and trigger instant settlements.

2.2 General Use Case Diagram



This section details the primary interactions between the identified actors and the AI-Powered Insurance Simulator Platform. The architecture is deliberately structured to separate public onboarding, user-centric financial simulations, administrative oversight, and the automated data-driven core of the parametric engine.

2.2.1 Guest Operations

Guest operations focus on platform discovery and user acquisition.

- **Browse Platform Information:** Unauthenticated individuals can explore educational content regarding risk management, micro-insurance concepts, and how the simulation operates.
- **Register Account:** Guests initiate the onboarding process to create a secure local account, enabling them to transition into authenticated Users.

2.2.2 User Operations

The User acts as the primary consumer of the simulation, engaging directly with the risk assessment and virtual financial tools.

- **Log In & Manage Profile:** Standard identity verification and account management functionalities.
- **Purchase Virtual Policy:** Users can select and acquire short-term parametric insurance policies. To enforce the realism of the financial simulation, this use case strictly includes the **Deduct Virtual Currency** sub-process, ensuring wallet balances are updated atomically upon purchase.
- **Run Risk Simulation:** Users interact with the AI-driven engine to evaluate historical data or predictive models, helping them determine the statistical viability and cost-effectiveness of a specific policy before purchasing.
- **View Claim History:** Users can monitor their active policies, past purchases, and the status of any automated payouts.

2.2.3 Administrative Operations

Administrators maintain the platform's stability, user base, and the underlying business logic of the virtual marketplace.

- **Manage User Accounts:** Administrators control account lifecycles, role assignments, and access permissions.
- **Configure Policy Marketplace:** Administrators define and update the catalog of available insurance products, setting baseline premiums, coverage limits, and the specific trigger conditions for each policy type.
- **Monitor Parametric Engine:** Administrators oversee the operational health of the automated claims system, ensuring that external data feeds are processed correctly and rule sets are executing as expected.

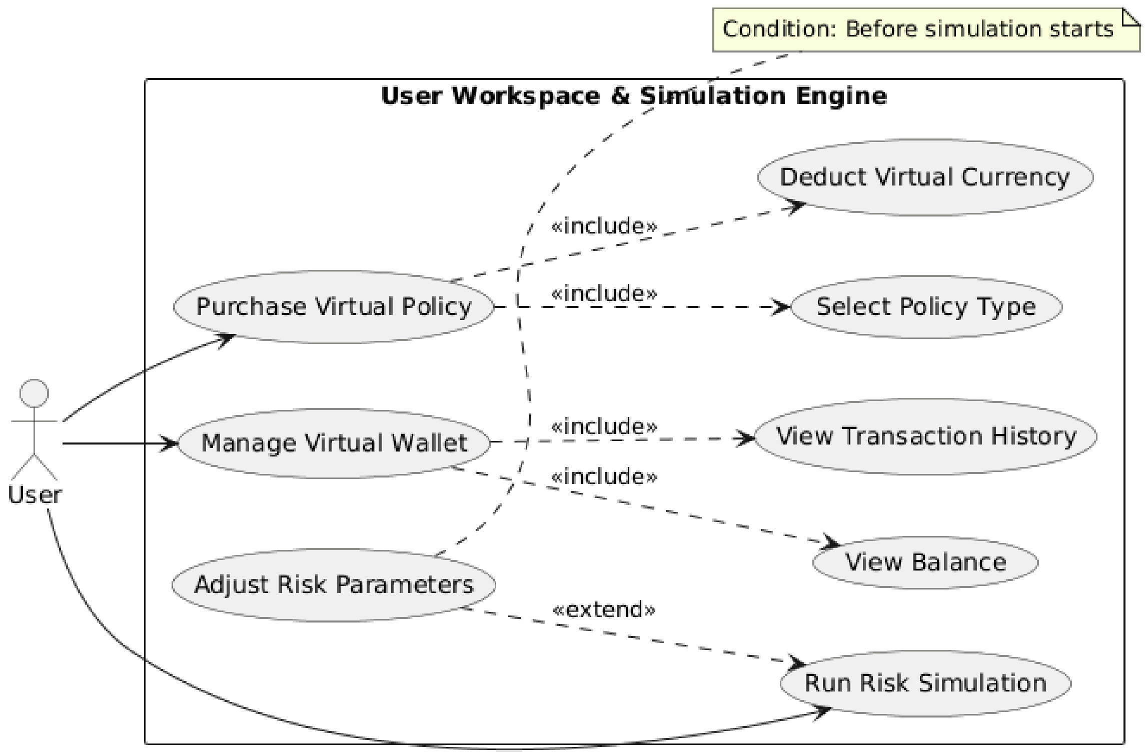
2.2.4 Automated Parametric Engine Flow

This flow represents the technical differentiator of the platform, highlighting the zero-touch settlement characteristic of parametric insurance.

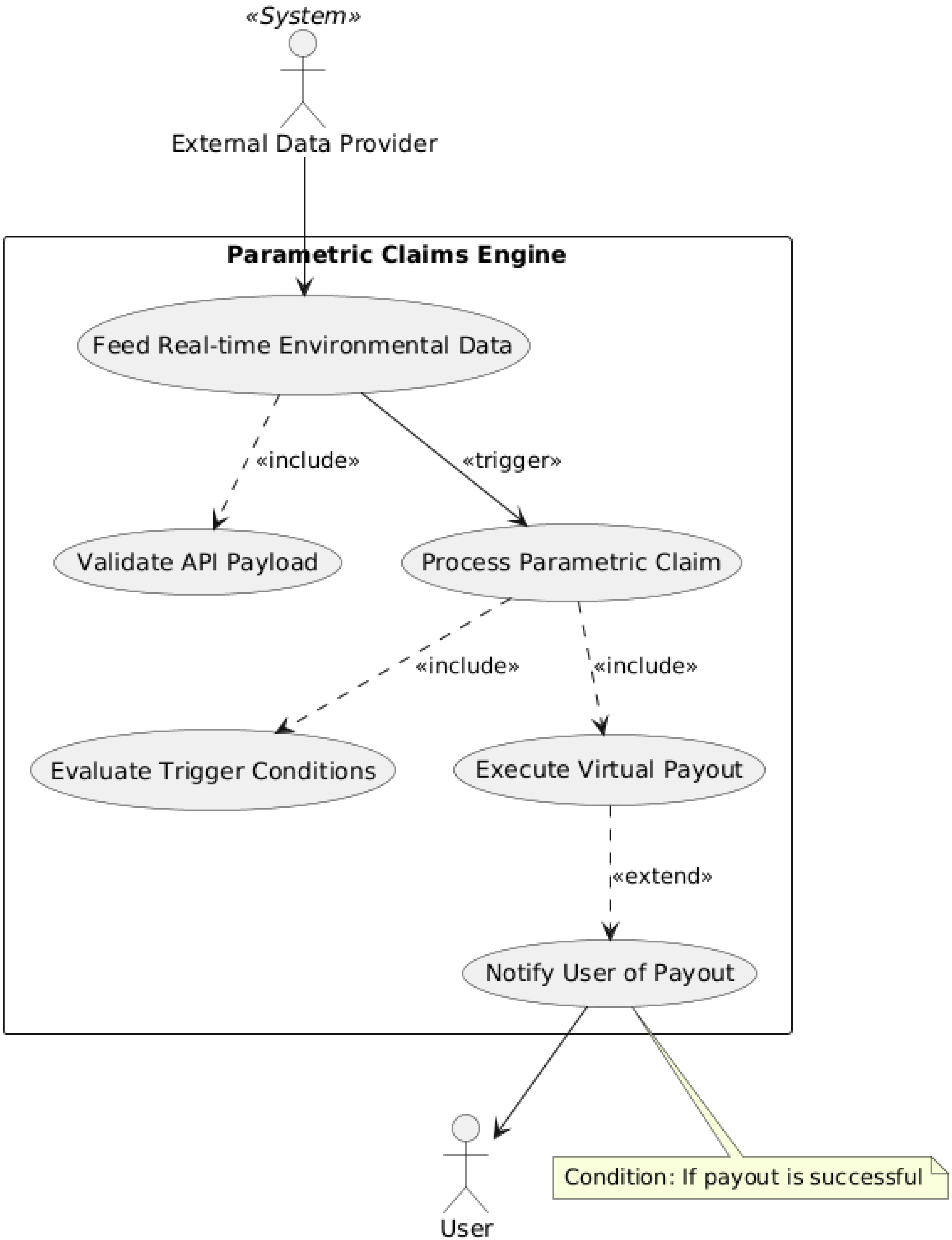
- **Provide Real-time Trigger Data:** The External Data Provider continuously supplies the platform with required external metrics via API integrations.
- **Automated Claim Processing:** Rather than relying on manual user submissions, this core system function is actively triggered by the incoming data feeds. If the data matches the predefined risk parameters of an active policy, the system automatically evaluates the claim and strictly includes the **Credit Virtual Wallet** use case to instantaneously disburse the simulated settlement to the affected User.

2.3 Overall System Use Case Description

2.3.1 User Workspace and Simulation Engine Subsystem



2.3.2 Automated Parametric Claims Engine Subsystem



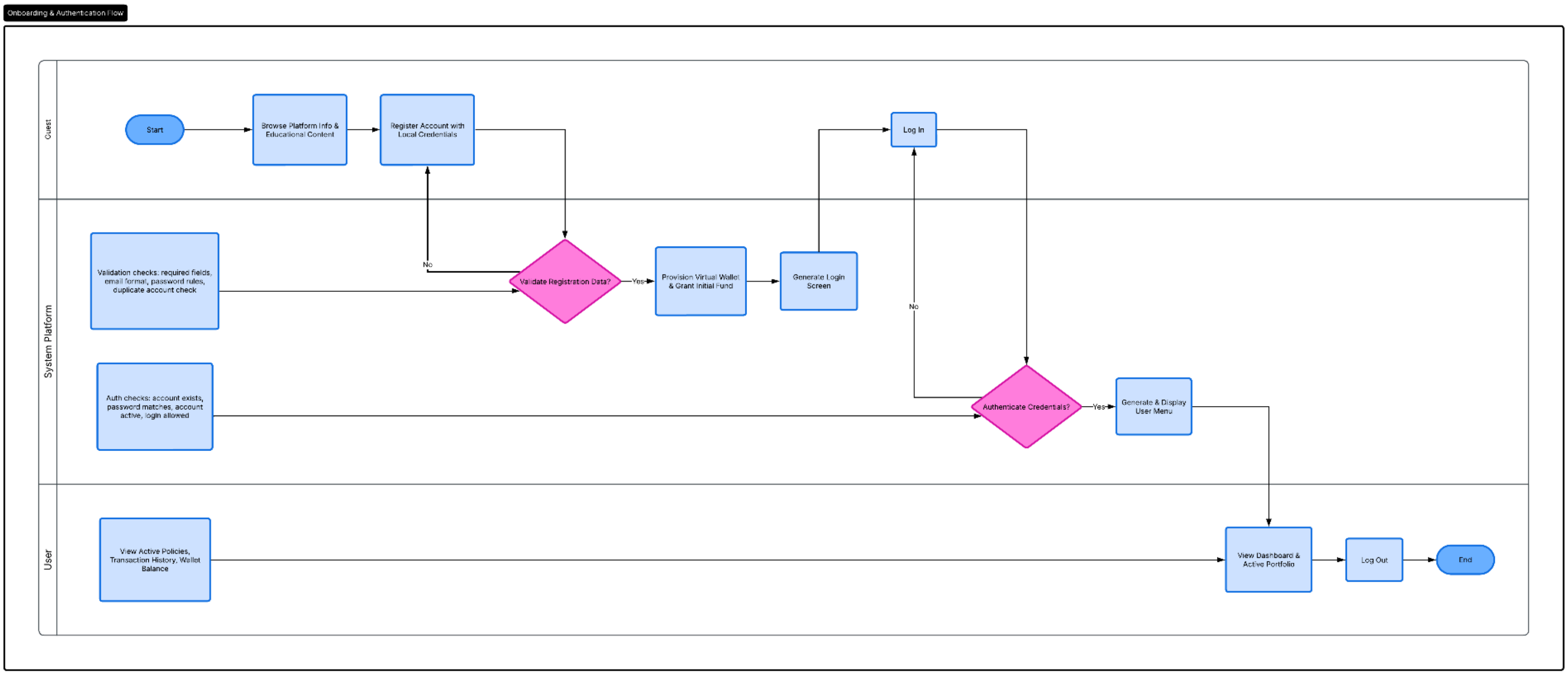
2.4 Core Business Processes

This section outlines the step-by-step operational workflows of the platform, categorized by the primary actors and their system journeys.

2.4.1. Guest Onboarding and User Operational Process

This process defines the end-to-end journey of an individual discovering the platform, acquiring simulation currency, and participating in the virtual marketplace.

- **Platform Discovery:** The Guest accesses the public landing page to review educational materials regarding parametric insurance concepts.
- **Account Registration & Provisioning:** The Guest registers an account using local credentials. Upon successful email verification, the system automatically transitions the Guest to a User role and provides a "Virtual Wallet," crediting it with an initial balance of virtual currency.
- **Marketplace Browsing:** The User navigates the active policy marketplace, reviewing available short-term insurance products.
- **Policy Acquisition:** The User selects a policy. The system calculates the required premium, deducts the amount directly from the User's Virtual Wallet, and issues an active virtual contract tied to the User's portfolio.
- **Portfolio Monitoring:** The User accesses their dashboard to track active policies, view transaction history, and monitor any automated payouts triggered by external events.

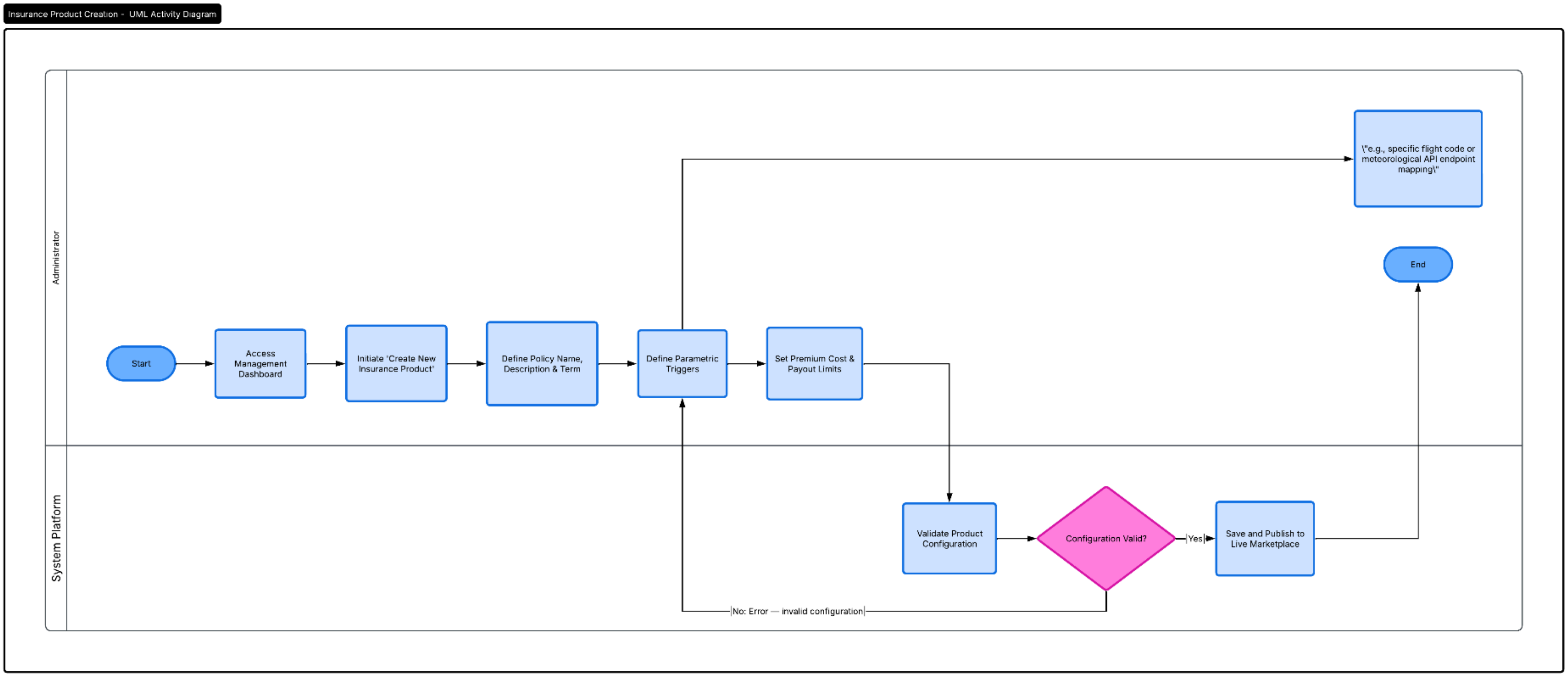


Onboarding & Authentication Flow
Guest
System Platform
User
Start
Browse Platform Info & Educational Content
Register Account with Local Credentials
Log In
Validation checks: required fields, email format, password rules, duplicate account check
No
Validate Registration Data?
Yes
Provision Virtual Wallet & Grant Initial Fund
Generate Login Screen
No
Auth checks: account exists, password matches, account active, login allowed
Authenticate Credentials?
Yes
Generate & Display User Menu
View Active Policies, Transaction History, Wallet Balance
View Dashboard & Active Portfolio
Log Out
End

2.4.2. Administrator Policy Creation and Deployment Process

This workflow outlines how the platform operators construct and introduce new parametric insurance products into the virtual ecosystem.

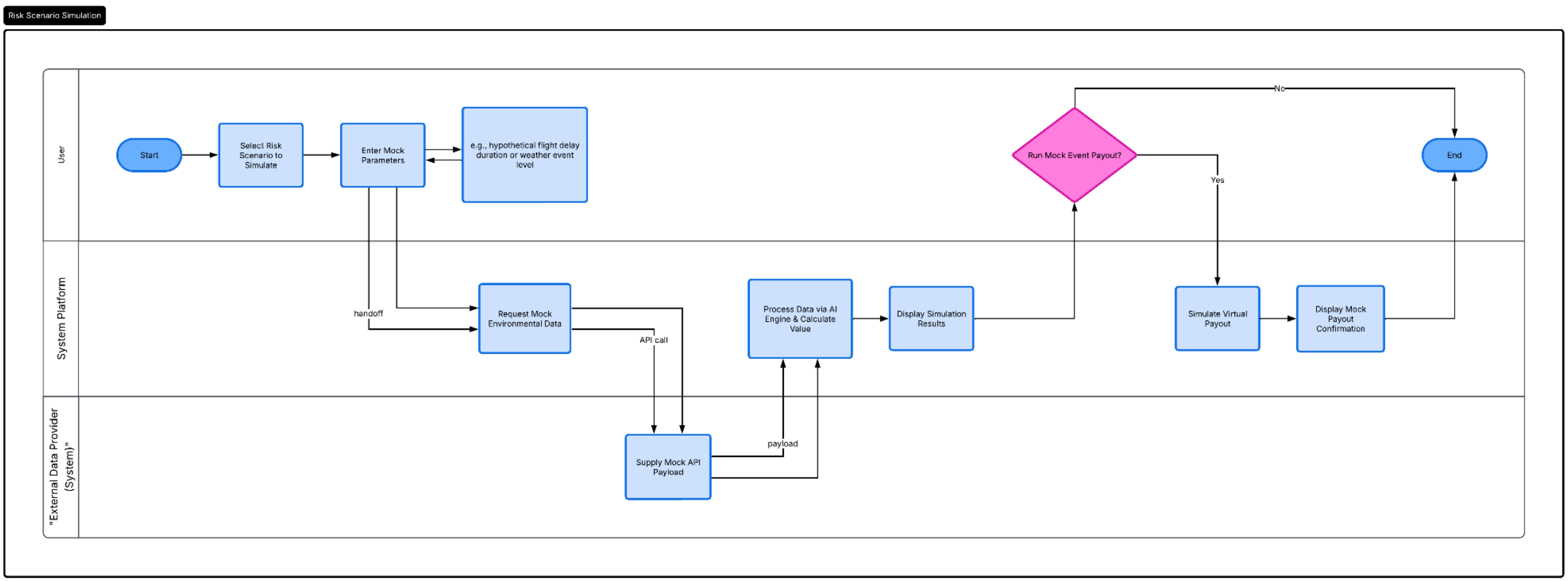
- **Product Initialization:** The Administrator accesses the management dashboard and initiates the creation of a new insurance product, defining core metadata
- **Parametric Trigger Configuration:** The Administrator defines the exact mathematical conditions that constitute a valid claim
- **Data Source Integration:** The Administrator maps the newly created triggers to a specific External Data Provider API
- **Financial Modeling:** The Administrator configures the baseline premium costs, maximum payout limits, and risk pooling margins.
- **Marketplace Deployment:** The system validates the configuration and publishes the new policy to the live virtual marketplace, making it immediately available for User simulation and purchase.


Insurance Product Creation - UML Activity Diagram
Administrator
System Platform
Start
Access Management Dashboard
Initiate 'Create New Insurance Product'
Define Policy Name, Description & Term
Define Parametric Triggers
Set Premium Cost & Payout Limits
\"e.g., specific flight code or meteorological API endpoint mapping\"
End
Validate Product Configuration
Configuration Valid?
Yes
Save and Publish to Live Marketplace
No: Error — invalid configuration

2.4.3. AI-Driven Risk and Claim Simulation Process

This core educational feature allows Users to test scenarios and understand the underlying risk calculations before committing virtual currency.

- **Scenario Selection:** The User inputs specific parameters for a desired policy
- **Data Retrieval & Inference:** The system's simulation engine queries historical datasets and utilizes AI-driven predictive models to calculate the probability of the risk event occurring.
- **Financial Projection:** The system calculates the estimated premium versus the potential payout, displaying the expected value of the policy.
- **Claim Simulation:** The User can manually trigger a "mock event" (simulating a delayed flight) to observe how the parametric engine instantly executes a virtual payout without requiring a manual claim submission


Risk Scenario Simulation
User
System Platform
"External Data Provider (System)"
Start
Select Risk Scenario to Simulate
Enter Mock Parameters
e.g., hypothetical flight delay duration or weather event level
handoff
Request Mock Environmental Data
API call
Supply Mock API Payload
payload
Process Data via AI Engine & Calculate Value
Display Simulation Results
Run Mock Event Payout?
No
Yes
Simulate Virtual Payout
Display Mock Payout Confirmation
End

3 Use Case Specifications

3.1 Use case UC001 “Register Account”

<table>
<tr><td>Use case code</td><td>UC001</td><td>Use case name</td><td>Register Account</td></tr>
<tr><td>Actor</td><td colspan="3">Guest</td></tr>
<tr><td>Pre-condition</td><td colspan="3">Actor does not have an account</td></tr>
<tr><td>Main Flow</td><td colspan="3">
<table>
<tr><th>No.</th><th>Actor</th><th>Action</th></tr>
<tr><td>1.</td><td>User</td><td>enters registration information (name, email, password, phone)</td></tr>
<tr><td></td><td>System</td><td>validates input</td></tr>
<tr><td>2.</td><td>System</td><td>finishes KYC process</td></tr>
<tr><td>3.</td><td>System</td><td>creates new user record in USERS table</td></tr>
<tr><td>4.</td><td>System</td><td>creates corresponding virtual wallet</td></tr>
<tr><td>5.</td><td>System</td><td>confirms successful registration</td></tr>
</table>
</td></tr>
<tr><td>Alternative Flow</td><td colspan="3">
<table>
<tr><th>No.</th><th>Actor</th><th>Action</th></tr>
<tr><td>2a.</td><td>System</td><td>Invalid email format → show error</td></tr>
<tr><td>3a</td><td>System</td><td>Email already exists → reject registration</td></tr>
</table>
</td></tr>
</table>

*Input and Output of use case:

No.	Input	Output
1.	Email	Authentication token
2.	Password	
3.	Name	
4.	Phone	

3.2 Use case UC002 – “Login”

Use case Code	UC002	**Use case name**	Login

Actor	User
Pre-condition	User has a valid account

Main Flow

No.	Actor	Action
1.	User	enters email and password
2.	System	verifies credentials
3.	System	generates authentication token
4.	User	is logged in

Alternative Flow:

No.	Actor	Action
2a	System	Invalid credentials → show error

*Input and Output of use case:

No.	Input	Output
3.	Email	Authentication token
4.	Password	

3.3 Use case UC003 "Browse Platform Information"

Use case code	UC003	**Use case name**	Browse Platform Information
Actor	Guest		
Pre-condition	No		

Main Flow

No.	Actor	Action
1.	Guest	accesses landing page
2.	System	displays educational content about insurance and simulation

Alternative Flow

No.	Actor	Action

3.4 Use case UC004 “View Insurance Products ”

<table>
<tr><td>Use case code</td><td>UC004</td><td>Use case name</td><td>View Insurance Products</td></tr>
<tr><td>Actor</td><td colspan="3">User, Guest</td></tr>
<tr><td>Pre-condition</td><td colspan="3">No</td></tr>
<tr><td>Main Flow</td><td colspan="3">
<table>
<tr><th>No.</th><th>Actor</th><th>Action</th></tr>
<tr><td>1.</td><td>User</td><td>accesses product list</td></tr>
<tr><td>2.</td><td>System</td><td>retrieves products</td></tr>
<tr><td>3.</td><td>System</td><td>displays available insurance products</td></tr>
</table>
</td></tr>
<tr><td>Alternative Flow</td><td colspan="3">
<table>
<tr><th>No.</th><th>Actor</th><th>Action</th></tr>
</table>
</td></tr>
</table>

* Output of use case:

No.	Output
1.	List of insurance products

3.5 Use case UC005 “View Insurance Product Details ”

<table>
<tr><td>Use case code</td><td>UC005</td><td>Use case name</td><td>View Insurance Product Details</td></tr>
<tr><td>Actor</td><td colspan="3">User, Guest</td></tr>
<tr><td>Pre-condition</td><td colspan="3">No</td></tr>
<tr><td>Main flow</td><td colspan="3">
<table>
<tr><th>STT</th><th>Thực hiện bởi</th><th>Hành động</th></tr>
<tr><td>1.</td><td>User</td><td>selects a product</td></tr>
<tr><td>2.</td><td>System</td><td>retrieves product details</td></tr>
<tr><td>3.</td><td>System</td><td>displays: description, premium, payout, trigger conditions</td></tr>
</table>
</td></tr>
</table>

*Input and Output of use case:

No.	Input	Output
1.	product_id	product detail

3.6 Use case UC006 “Purchase Insurance Policy ”

Use case code	UC006	**Use case name**	Purchase Insurance Policy
Actor	User		
Pre-condition	User is logged in and has sufficient balance		

	No.	**Actor**	**Action**
Main Flow	1.	User	selects product
	2.	System	displays policy details
	3.	User	confirms purchase
	4.	System	checks wallet balance
	5.	System	Include → Deduct Virtual Currency
	6.	System	creates new record in POLICIES
	7.	System	confirms purchase
Alternative Flow	4a	System	Insufficient balance → transaction rejected
	6a	System	Database error → rollback transaction

*Input and Output of use case:

No.	**Input**	**Output**
1.	user_id	policy_id
2.	product_id	transaction record

3.7 Use case UC007 “View Purchased Policies”

Use case code	UC007	**Use case name**	View Purchased Policies
Actor	User		
Main Flow			

No.	**Actor**	**Action**
1.	User	requests policy list
2.	System	retrieves policies
3.	System	displays policies

3.8 Use case UC008 “Run Insurance Simulation”

Use case code	UC008	**Use case name**	Run Insurance Simulation
Actor	User		
Main Flow	(see table below)		

No.	Actor	Action
1.	User	selects simulation mode
2.	User	inputs parameters
3.	System	calculates risk, premium, payout
4.	System	stores simulation result
5.	System	displays recommendation

*Input and Output of use case:

No.	Input	Output
1.	input_parameters	simulation result

3.9 Use case UC009 “AI Chatbot Consultation”

Use case code	UC009	**Use case name**	AI Chatbot Consultation
Actor	User		
Main Flow	(see table below)		

No.	Actor	Action
1.	User	sends query
2.	System	forwards to AI engine (Gemini)
3.	AI	generates response
4.	System	returns response to user

3.10 Use case UC010 “Detect External Event”

Use case code	UC010	**Use case name**	Detect External Event
Actor	System		
Main Flow			

No.	Actor	Action
1.	System	calls external APIs
2.	System	receives event data
3.	System	stores event
4.	System	updates status

3.11 Use case UC011 “Process Claim Automatically”

Use case code	UC011	**Use case name**	Process Claim Automatically
Actor	System		
Pre-condition	Event detected		
Main Flow			

No.	Actor	Action
1.	System	matches event with policies
2.	System	evaluates trigger_conditions
3.	System	creates claim record
4.	System	links event and claim
5.	System	marks claim as approved

Alternative Flow

No.	Actor	Action
2a.	System	Conditions not met → no claim created

3.12 Use case UC012 “Credit Wallet After Claim”

<table>
<tr><td>Use case code</td><td>UC012</td><td>Use case name</td><td>Credit Wallet After Claim</td></tr>
<tr><td>Actor</td><td colspan="3">System</td></tr>
<tr><td>Main Flow</td><td colspan="3">
<table>
<tr><th>No.</th><th>Actor</th><th>Action</th></tr>
<tr><td>1.</td><td>System</td><td>calculates claim amount</td></tr>
<tr><td>2.</td><td>System</td><td>updates wallet</td></tr>
<tr><td>3.</td><td>System</td><td>records transaction</td></tr>
<tr><td>4.</td><td>System</td><td>updates claim status</td></tr>
</table>
</td></tr>
</table>

3.13 Use case UC013 “View Transaction History”

<table>
<tr><td>Use case code</td><td>UC013</td><td>Use case name</td><td>View Transaction History</td></tr>
<tr><td>Actor</td><td colspan="3">User</td></tr>
<tr><td>Main Flow</td><td colspan="3">
<table>
<tr><th>No.</th><th>Actor</th><th>Action</th></tr>
<tr><td>1.</td><td>User</td><td>requests transaction history</td></tr>
<tr><td>2.</td><td>System</td><td>retrieves data</td></tr>
<tr><td>3.</td><td>System</td><td>displays list</td></tr>
</table>
</td></tr>
</table>

3.14 Use case UC014 “Manage Insurance Products”

<table>
<tr><td>Use case code</td><td>UC014</td><td>Use case name</td><td>Manage Insurance Products</td></tr>
<tr><td>Actor</td><td colspan="3">Admin</td></tr>
<tr><td>Main Flow</td><td colspan="3">
<table>
<tr><th>No.</th><th>Actor</th><th>Action</th></tr>
<tr><td>1.</td><td>Admin</td><td>creates/updates/deletes products</td></tr>
</table>
</td></tr>
</table>

No.	Actor	Action
2.	System	updates database

3.15 Use case UC015 “Manage Product Rules”

Use case code	UC015	Use case name	Manage Product Rules
Actor	Admin		
Main Flow			

No.	Actor	Action
1.	Admin	defines rules
2.	System	stores rules

3.16 Use case UC016 “Monitor Claims”

Use case code	UC016	Use case name	Monitor Claims
Actor	Admin		
Main Flow			

No.	Actor	Action
1.	User	views claims
2.	System	retrieves claims
3.	System	displays claim status

*Input and Output of use case:

No.	Input	Output
5.	Email	Authentication token
6.	Password	

3.17 Use case UC017 "Manage Profile"

Use case code		**Use case name**	Manage Profile
Actor	User		
Main Flow			

No.	Actor	Action
1.	User	updates profile information
2.	System	validates input
3.	System	saves changes

3.18 Use case UC018 "Manage User Accounts"

Use case code	UC018	**Use case name**	Manage User Accounts
Actor	Admin		
Main Flow			

No.	Actor	Action
1.	Admin	views user list
2.	Admin	updates roles / status
3.	System	saves changes

3.19 Use case UC019 "Monitor Parametric Engine"

Use case code	UC019	**Use case name**	Monitor Parametric Engine
Actor	Admin		
Main Flow			

No.	Actor	Action
1.	Admin	views system status
2.	System	displays data flow & processing logs

4 Other Non-functional Requirements

4.1 Performance & Scalability Requirements

The system must ensure high performance and the ability to scale as the number of users and data volume increases.

- API response time must not exceed **2 seconds** under normal load conditions
  (applies to: Login, View Products, Run Simulation)
- Automated claim processing must occur in **less than 5 seconds** after receiving data from the External Data Provider
  (applies to: Detect External Event, Process Claim Automatically)
- The system must support concurrent users without significant performance degradation, especially during:
  - policy purchase
  - simulation execution
- The system must support **horizontal scalability** using containerization to handle increasing demand

4.2 Security & Access Control Requirements

The system must ensure data security and enforce strict access control using a role-based model.

- All users must be authenticated using **token-based authentication**
  (applies to: Register Account, Login)
- User passwords must be securely hashed before being stored in the database
- The system must implement **Role-Based Access Control**:
  - Guest: view public information only
  - User: purchase policies, run simulations
  - Admin: manage system resources and configurations
- All protected APIs must require a valid authentication token
- External API keys must be securely stored and must not be exposed to the client side

4.3 Reliability & Data Consistency Requirements

The system must maintain stable operation and ensure data consistency, especially for financial simulation transactions.

- Critical operations must satisfy **ACID properties**
  (applies to: Purchase Policy, Credit Wallet After Claim)
- The system must implement a **retry mechanism** for failed external API calls
  (applies to: Detect External Event)
- In case of system or database failure:
  - transactions must be rolled back
  - inconsistent states must be prevented
- All system errors and important events must be **logged** for monitoring and debugging purposes

4.4 Usability & System Integration Requirements

The system must provide a user-friendly interface and integrate effectively with external components.

- The user interface must be:
  - simple and intuitive
  - capable of clearly presenting simulation results
    (applies to: Run Insurance Simulation)
- The system must provide clear feedback messages for:
  - successful operations
  - failed operations
- The system must integrate with:
  - External Data Providers for automated claim processing
  - AI engine for chatbot consultation *(applies to: AI Chatbot Consultation)*
- The AI chatbot must respond within a reasonable time (less than **3 seconds**) and handle common insurance-related queries effectively